● 2026 한경국립대학교 컴퓨터응용수학부 컴퓨터 공학전공 졸업 전시회

제 24회 졸업전시회
작품집

AI·IoT 기반 관상어용 스마트 어항 시스템

< 김서원, 유승엽, 이다경 >

한경국립대학교
Hankyong National University

2026학년도 캡스톤디자인 최종보고서

AI·IoT 기반 관상어용 스마트 어항 시스템

수행분야	졸업작품/경진대회			
지도교수	성 명	김미희교수	확 인	
팀 원	학 번	성 명	역할/주요업무	
	2023620008	김서원	IOT·프론트/팀장	
	2021810042	유승엽	백엔드·프론트/팀원	
	2023810046	이다경	AI·프론트/팀원	

한경국립대학교 컴퓨터응용수학부 컴퓨터 공학전공

목 차

1. 서 론
1.1. 개발 배경(주제 선정 이유)
1.2. 작품 개요
1.3. 선행 연구 및 작품관련 자료조사
1.4. 작품의 목표
1.5. 작품의 특징 및 기대효과
1.6. 팀원의 구성 및 역할 분담

2. 작품의 분석/설계
2.1. 설계개요(개념설계, 시스템 구성도)
2.2. UI 설계
2.3. 시스템 상세설계
2.4. 시스템 기능별 요약

3. 결론 및 향후과제
3.1. 작품보완점 및 목표구현 정도
3.2. 개발 환경 및 도구
3.3. 향후 발전 방향
3.4. 문제점 및 개선 방안

4. 실행 매뉴얼: 소스 코드 실행 환경 및 실행 방법

5. 사용자 매뉴얼: 작품의 사용 방법

부 록
R-1. 참고문헌 및 참고사이트
R-2. 프로그램 소스(소프트웨어 개발 시)

1. 서 론

1.1. 개발 배경(주제 선정 이유)

관상어를 키우기 위해서는 먹이 급여, 환수, 수온 및 수질 관리 등 지속적인 관리가 필요하다. 그러나 잦은 외출이나 바쁜 일상으로 어항을 꾸준히 관리하기 어렵고, 관상어의 건강 상태를 파악하는 데에도 어려움이 있다. 이러한 문제를 해결하기 위해 IoT와 AI 기술을 활용하여 사용자가 원격으로 어항을 관리하고 관상어의 상태를 확인할 수 있게 함으로써 어항 관리에 대한 부담을 줄이고 보다 편리하게 관리할 수 있도록 해준다.

1.2. 작품 개요

본 시스템은 IoT와 AI를 활용한 스마트 수조관리 시스템이다. 사용자는 모바일 앱을 통해 실시간 영상과 수온 및 수질을 확인하고, 원격으로 먹이 급여와 환수를 실행할 수 있다. 또한 AI를 활용하여 관상어의 어종과 개체 수를 인식하고, 움직임을 분석하여 활동성 이상 여부를 판단한다. 이외에도 어항 상태 기록 조회, 알림, 커뮤니티 및 AI 챗봇 기능을 제공한다.

1.3. 선행 연구 및 작품관련 자료조사

기능 ₩ 유사시스템	어항일지	어항일기	우리 시스템
관상어 메모 저장	O	x	O
수온·수질 관리 및 기록	△	△	O
원격 먹이 급여·환수	x	x	O
커뮤니티/ Q&A기능	O	x	O
실시간 영상 스트리밍	x	x	O
카메라로 관상어 양상 분석	x	x	O

개발 앱의 가장 큰 차별점은 사용자가 카메라로 어항을 실시간으로 관찰하고 원격으로 관리할 수 있으며, 어항의 상태가 자동으로 기록된다는 것이다. 기존 시스템은 사용자가 수온과 수질을 직접 측정하고 기록해야 하지만, 우리 시스템은 IoT 센서를 활용하여 데이터를 자동으로 수집하고 앱에서 확인할 수 있게 한다. 또한 AI를 활용하여 관상어의 어종과 개체 수를 인식하고 활동성 이상 여부를 분석한다. 이외에도 AI 챗봇과 사용자 간 커뮤니티 기능을 통해 어항 관리에 필요한 정보와 해결 방법을 쉽게 얻을 수 있도록 돕는다.

[1]어항 일지 - 어항 관리자
[2]어항일기 - 물생활 다이어리

1.4. 작품의 목표

본 시스템은 사용자가 시간과 장소에 관계없이 모바일 앱을 통해 어항을 원격으로 관리하고, AI를 활용하여 관상어의 상태를 확인할 수 있도록 하는 것을 목표로 한다. 이를 통해 지속적인 어항 관리가 어려운 사용자들의 부담을 줄이고, 관상어의 이상 징후를 조기에 파악하여 보다 편리하고 안정적인 관상어 사육 환경을 제공하고자 한다.

1.5. 작품의 특징 및 기대효과

예상사용자	제공 효과
관상어 사육 초보자	AI를 통한 활동성 분석으로 관상어의 상태를 쉽게 파악할 수 있다.
외출 및 출장이 잦은 사용자	실시간 영상 확인과 원격 먹이 급여 및 환수 기능으로 장소에 관계없이 어항을 관리할 수 있다.
관상어 정보를 공유하고 싶은 사용자	커뮤니티와 AI 챗봇을 통해 관상어에 대한 정보를 얻고 다른 사용자와 공유할 수 있다.

1.6. 팀원의 구성 및 역할 분담

기능	설명	담당자
로그인	해당 시스템을 이용하기 위해 사용자 인증을 실행한다.	유승엽
사용자 정보 관리	닉네임, 프로필 사진 및 어항 설정 기능을 구현한다.	유승엽
서버 및 데이터베이스	사용자 정보와 어항 데이터를 저장하고, 앱과 IoT 및 AI 서버 간의 통신을 처리한다.	유승엽
커뮤니티	게시글과 댓글을 통해 사용자 간 정보를 공유할 수 있도록 한다.	유승엽 김서원
IoT 센서 측정	센서를 통해 어항의 수온과 수질을 측정한다.	김서원
원격 먹이 급여 및 환수	먹이 급여와 환수를 원격으로 실행할 수 있도록 한다.	김서원

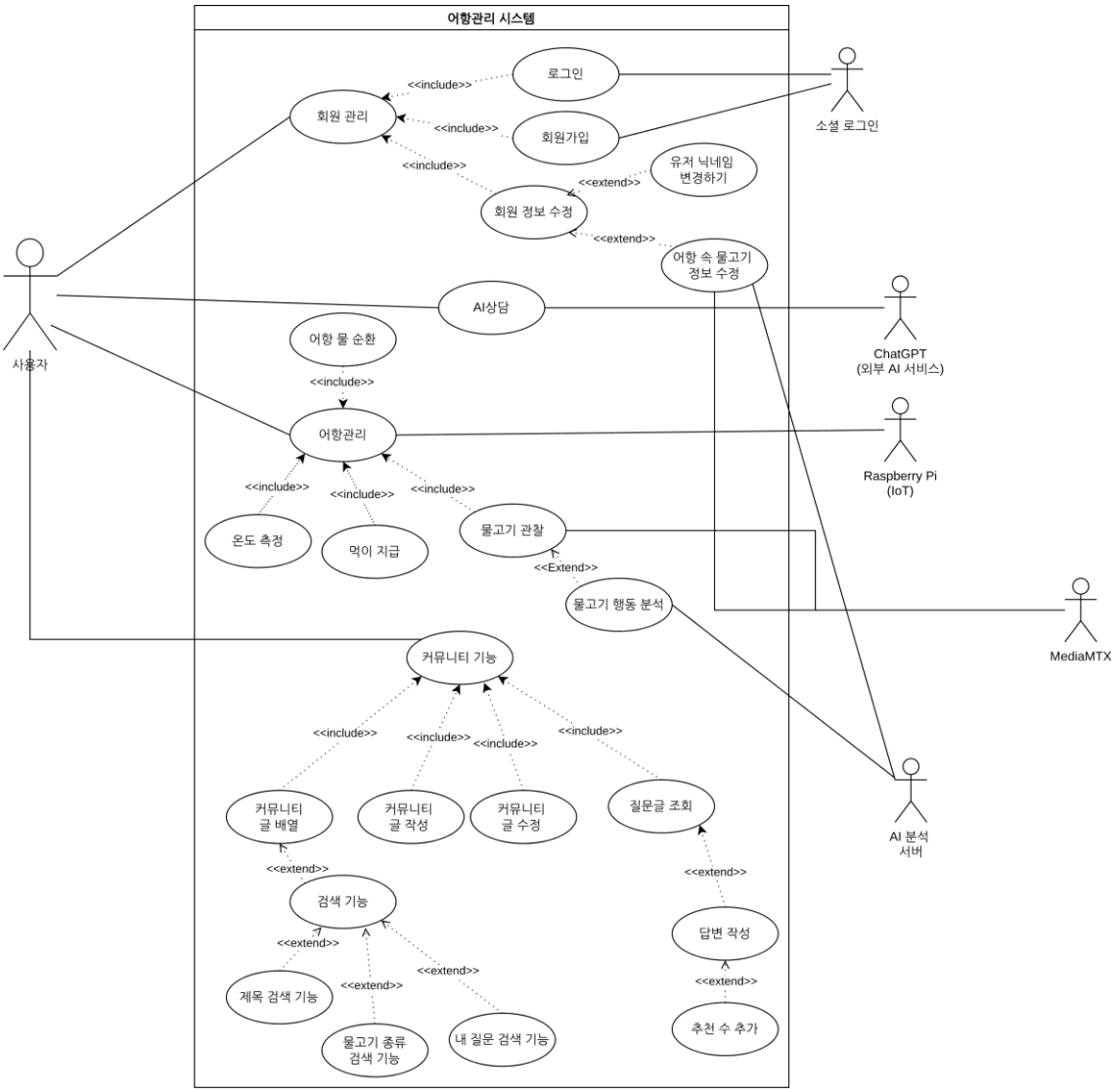
실시간 영상 스트리밍	WebRTC를 활용하여 어항 영상을 실시간으로 제공한다.	이다경
AI 어종 인식	AI 서버에서 YOLO를 활용하여 관상어의 종류와 개체 수를 인식한다.	이다경
AI 활동성 이상 탐지	AI 서버에서 객체 추적 및 SOM을 활용하여 관상어의 활동성 이상 여부를 분석한다.	이다경
수조 상태 모니터링	수온·수질 조회, 기록 및 이상 알림 기능을 제공한다.	김서원
이상 알람	온도나 수질에 이상이 생기면 사용자에게 이상 알람을 제공한다.	유승엽 김서원
AI 챗봇	LLM 기반 챗봇을 통해 관상어 관리 정보를 제공한다.	유승엽 김서원
일일 기록 확인	선택한 날짜의 어항 상태와 관리 기록을 확인한다.	유승엽 김서원

2. 작품의 분석/설계

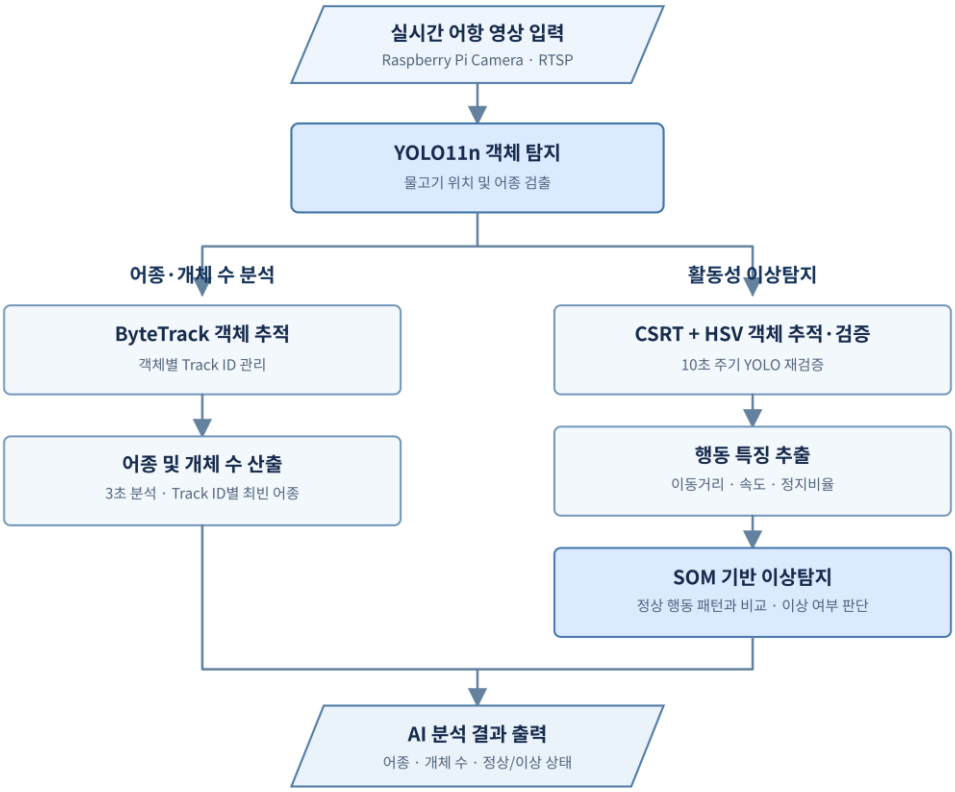
2.1. 설계개요(개념설계, 시스템 구성도)

SVG원본모음https://drive.google.com/file/d/1YRY-dnnr-gpUfBDOvX995mVlqBC_jvWI/view?usp=sharing

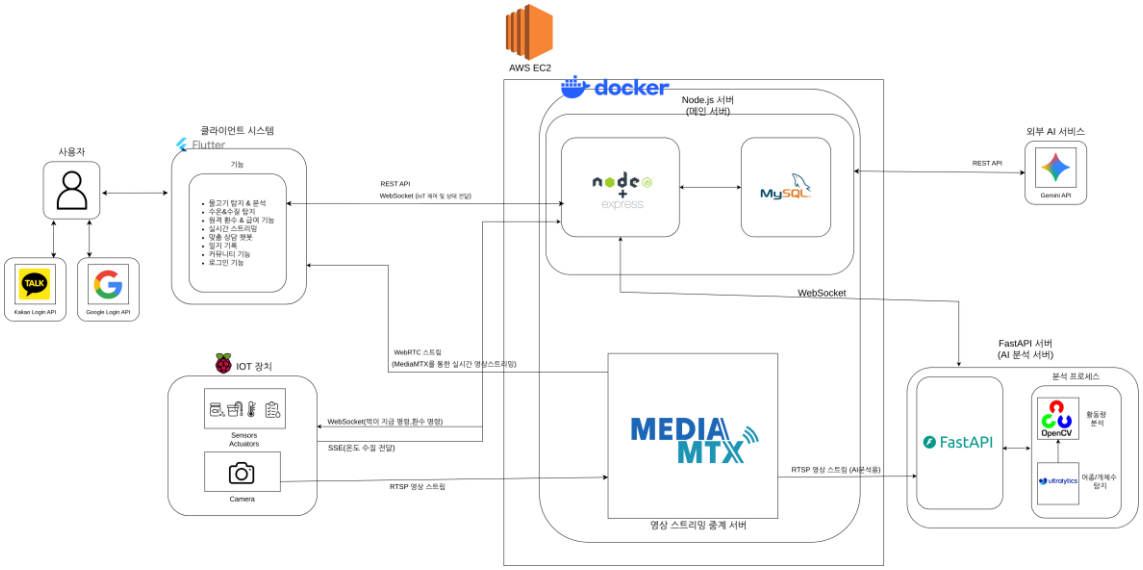
유스케이스 다이어그램



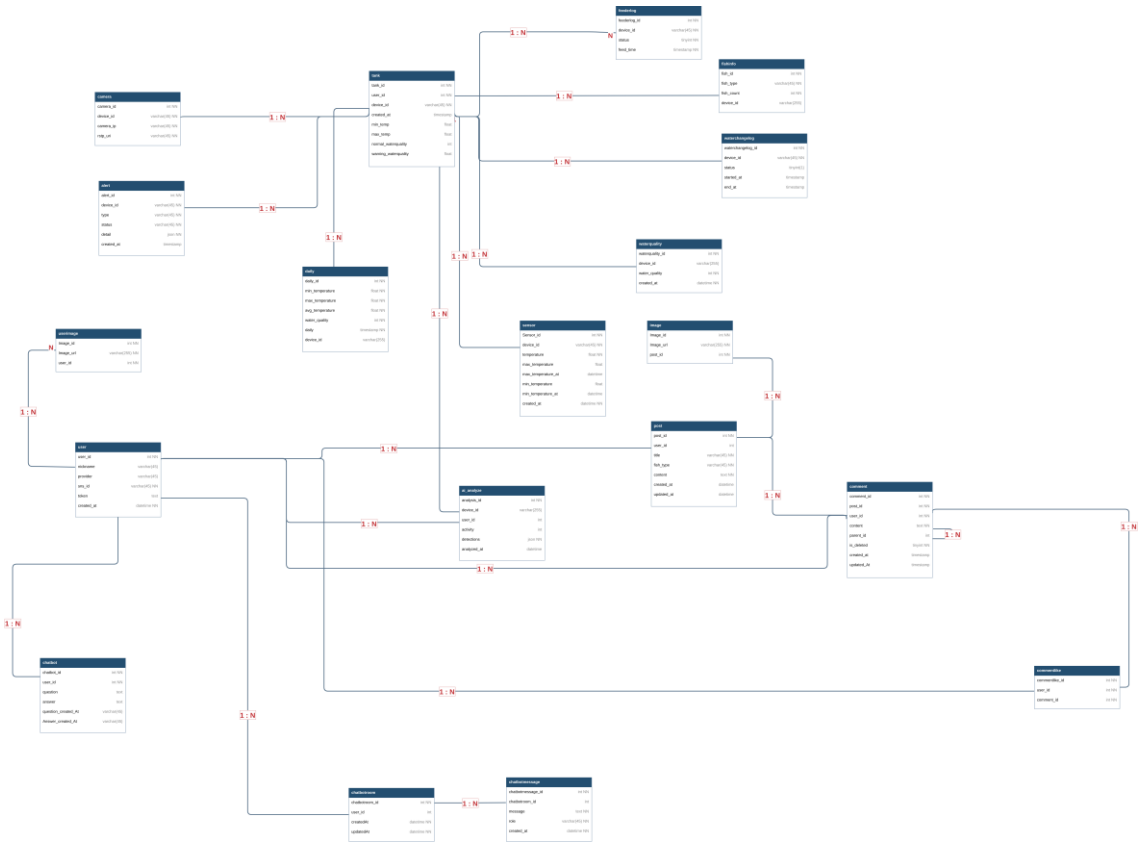
모델 아키텍쳐


실시간 어항 영상 입력
Raspberry Pi Camera · RTSP
YOLO11n 객체 탐지
물고기 위치 및 어종 검출
어종·개체 수 분석
활동성 이상탐지
ByteTrack 객체 추적
객체별 Track ID 관리
어종 및 개체 수 산출
3초 분석 · Track ID별 최빈 어종
CSRT + HSV 객체 추적·검증
10초 주기 YOLO 재검증
행동 특징 추출
이동거리 · 속도 · 정지비율
SOM 기반 이상탐지
정상 행동 패턴과 비교 · 이상 여부 판단
AI 분석 결과 출력
어종 · 개체 수 · 정상/이상 상태

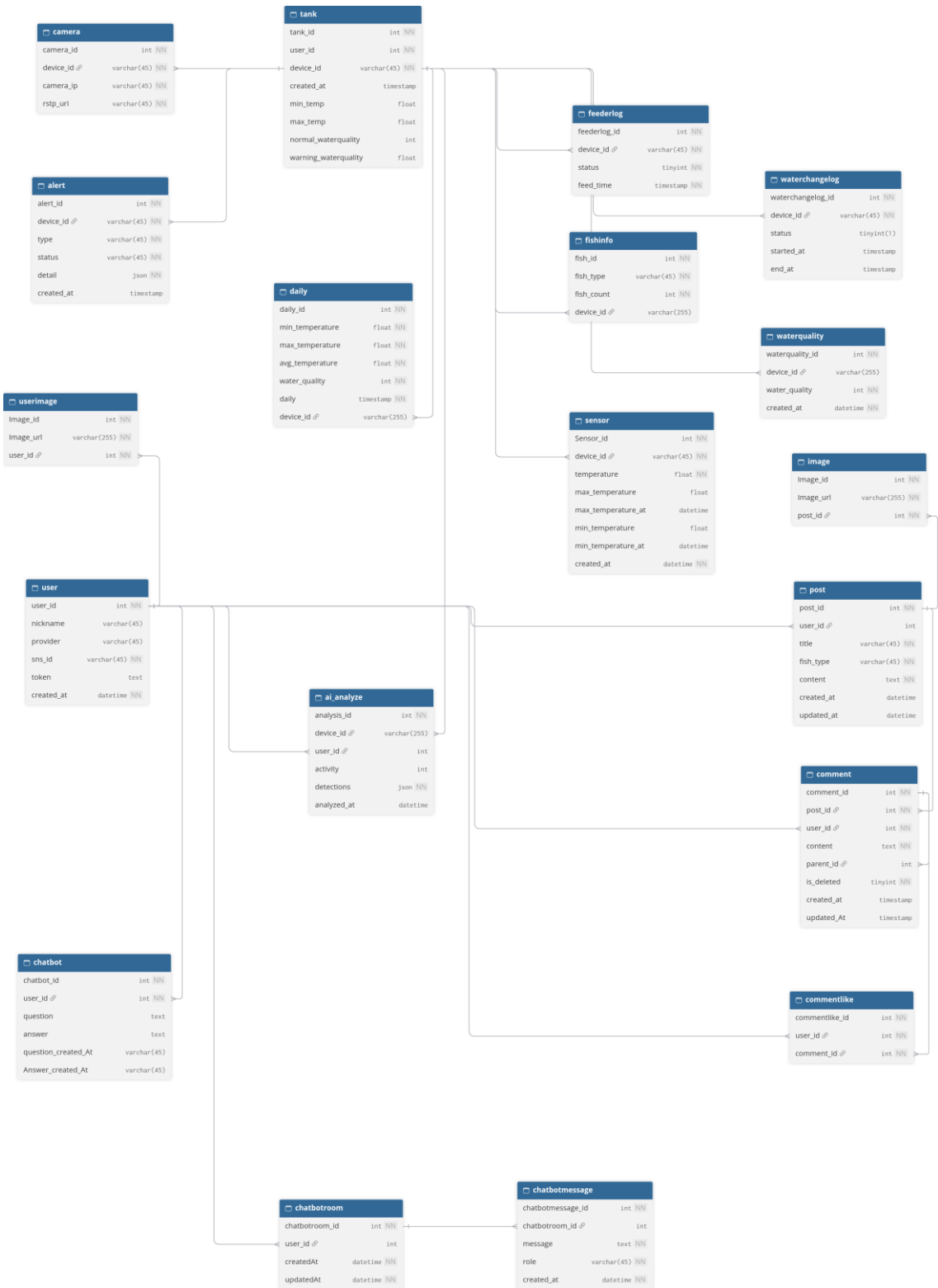
시스템 아키텍쳐


AWS EC2
docker
Node.js 서버
(메인 서버)
express
MySQL
외부 AI 서비스
REST API
Gemini API
사용자
클라이언트 시스템
Flutter
기능
• 물고기 탐지 & 분석
• 수온&수질 탐지
• 원격 환수 & 급여 기능
• 실시간 스트리밍
• 양총 상담 챗봇
• 일지 기록
• 커뮤니티 기능
• 로그인 기능
REST API
WebSocket (IoT 제어 및 상태 전달)
Kakao Login API
Google Login API
WebSocket
FastAPI 서버
(AI 분석 서버)
분석 프로세스
FastAPI
OpenCV
활동량 분석
ultralytics
어종/개체수 탐지
IOT 장치
WebRTC 스트림
(MediaMTX를 통한 실시간 영상스트리밍)
WebSocket(먹이 지급 명령,환수 명령)
SSE(온도 수질 전달)
Sensors
Actuators
Camera
RTSP 영상 스트림
MEDIA MTX
RTSP 영상 스트림 (AI분석용)
영상 스트리밍 중계 서버

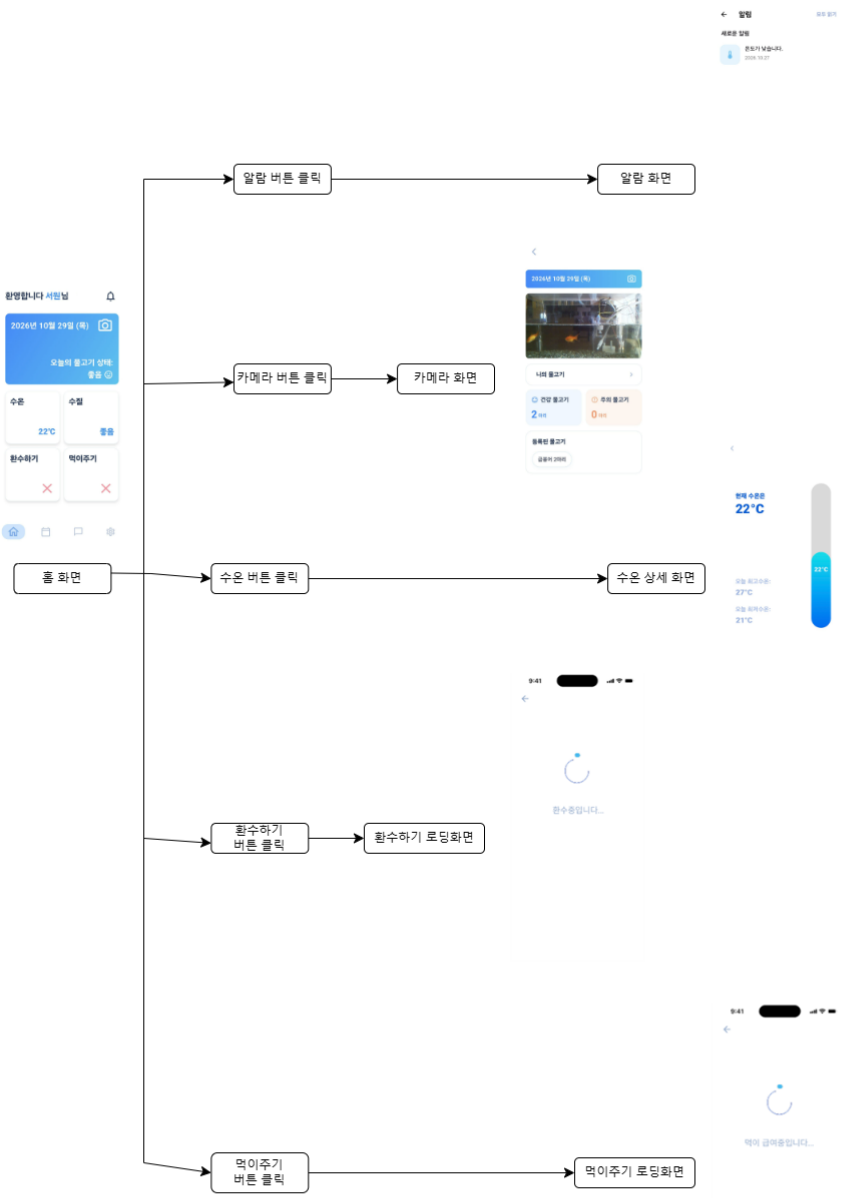
ERD

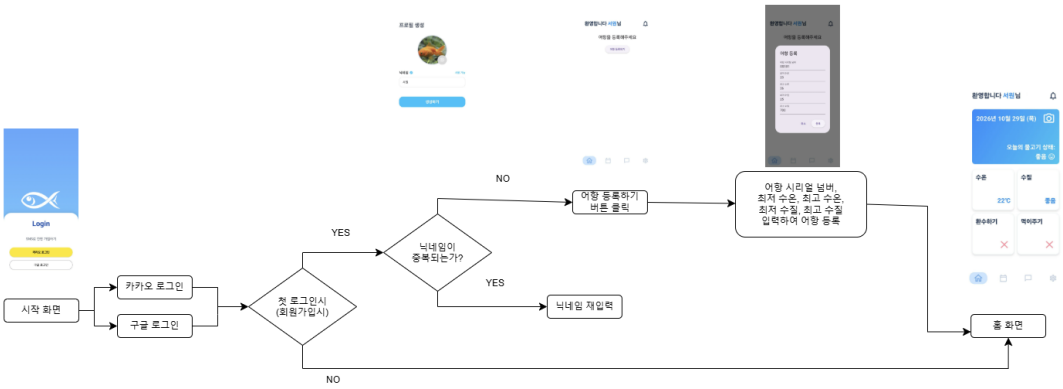


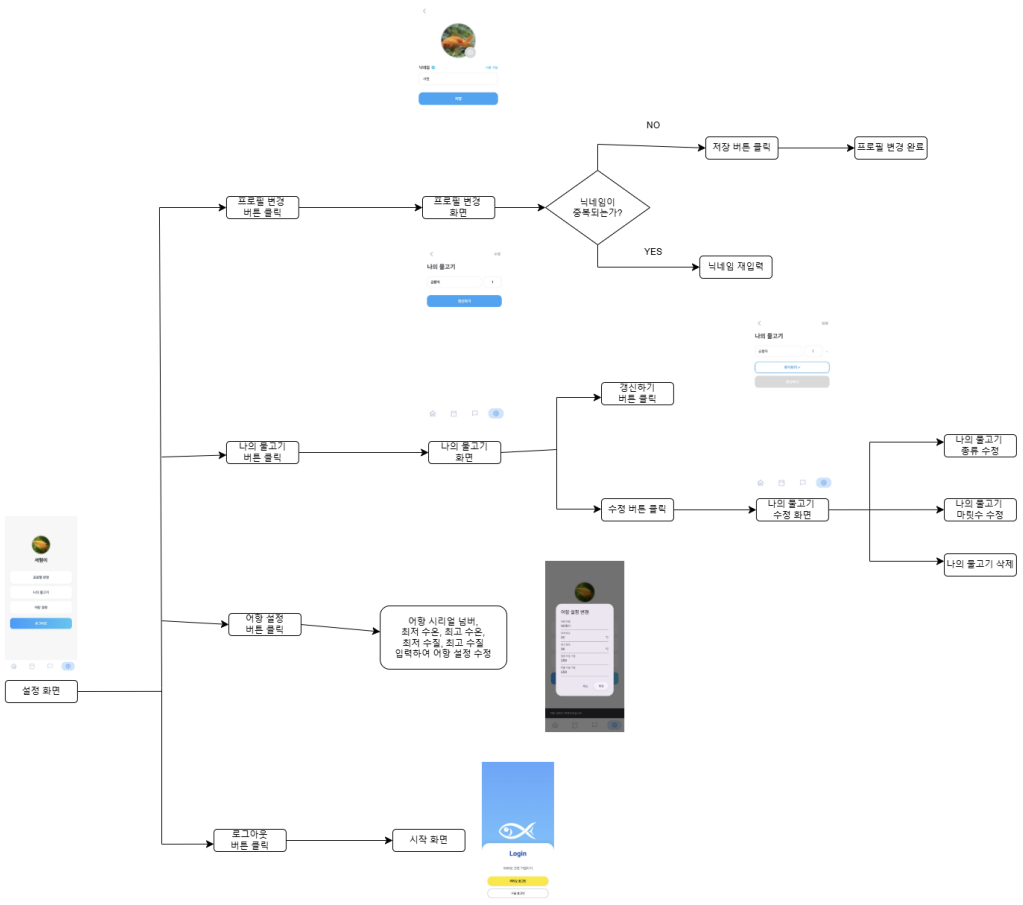
클래스 다이어그램

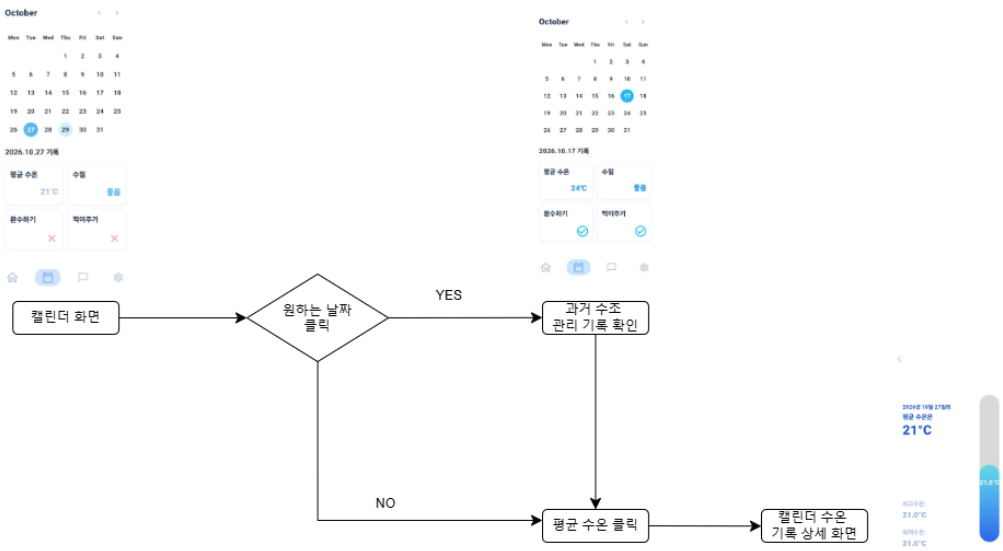

camera
camera_id int NN
device_id varchar(45) NN
camera_ip varchar(45) NN
rstp_url varchar(45) NN
tank
tank_id int NN
user_id int NN
device_id varchar(45) NN
created_at timestamp
min_temp float
max_temp float
normal_waterquality int
warning_waterquality float
feederlog
feederlog_id int NN
device_id varchar(45) NN
status tinyint NN
feed_time timestamp NN
waterchangelog
waterchangelog_id int NN
device_id varchar(45) NN
status tinyint(1)
started_at timestamp
end_at timestamp
alert
alert_id int NN
device_id varchar(45) NN
type varchar(45) NN
status varchar(45) NN
detail json NN
created_at timestamp
fishinfo
fish_id int NN
fish_type varchar(45) NN
fish_count int NN
device_id varchar(255)
daily
daily_id int NN
min_temperature float NN
max_temperature float NN
avg_temperature float NN
water_quality int NN
daily timestamp NN
device_id varchar(255)
waterquality
waterquality_id int NN
device_id varchar(255)
water_quality int NN
created_at datetime NN
userimage
Image_id int NN
Image_url varchar(255) NN
user_id int NN
sensor
Sensor_id int NN
device_id varchar(45) NN
temperature float NN
max_temperature float
max_temperature_at datetime
min_temperature float
min_temperature_at datetime
created_at datetime NN
image
Image_id int NN
Image_url varchar(255) NN
post_id int NN
user
user_id int NN
nickname varchar(45)
provider varchar(45)
sns_id varchar(45) NN
token text
created_at datetime NN
post
post_id int NN
user_id int
title varchar(45) NN
fish_type varchar(45) NN
content text NN
created_at datetime
updated_at datetime
ai_analyze
analysis_id int NN
device_id varchar(255)
user_id int
activity int
detections json NN
analyzed_at datetime
comment
comment_id int NN
post_id int NN
user_id int NN
content text NN
parent_id int
is_deleted tinyint NN
created_at timestamp
updated_At timestamp
chatbot
chatbot_id int NN
user_id int NN
question text
answer text
question_created_At varchar(45)
Answer_created_At varchar(45)
commentlike
commentlike_id int NN
user_id int NN
comment_id int NN
chatbotroom
chatbotroom_id int NN
user_id int
createdAt datetime NN
updatedAt datetime NN
chatbotmessage
chatbotmessage_id int NN
chatbotroom_id int
message text NN
role varchar(45) NN
created_at datetime NN

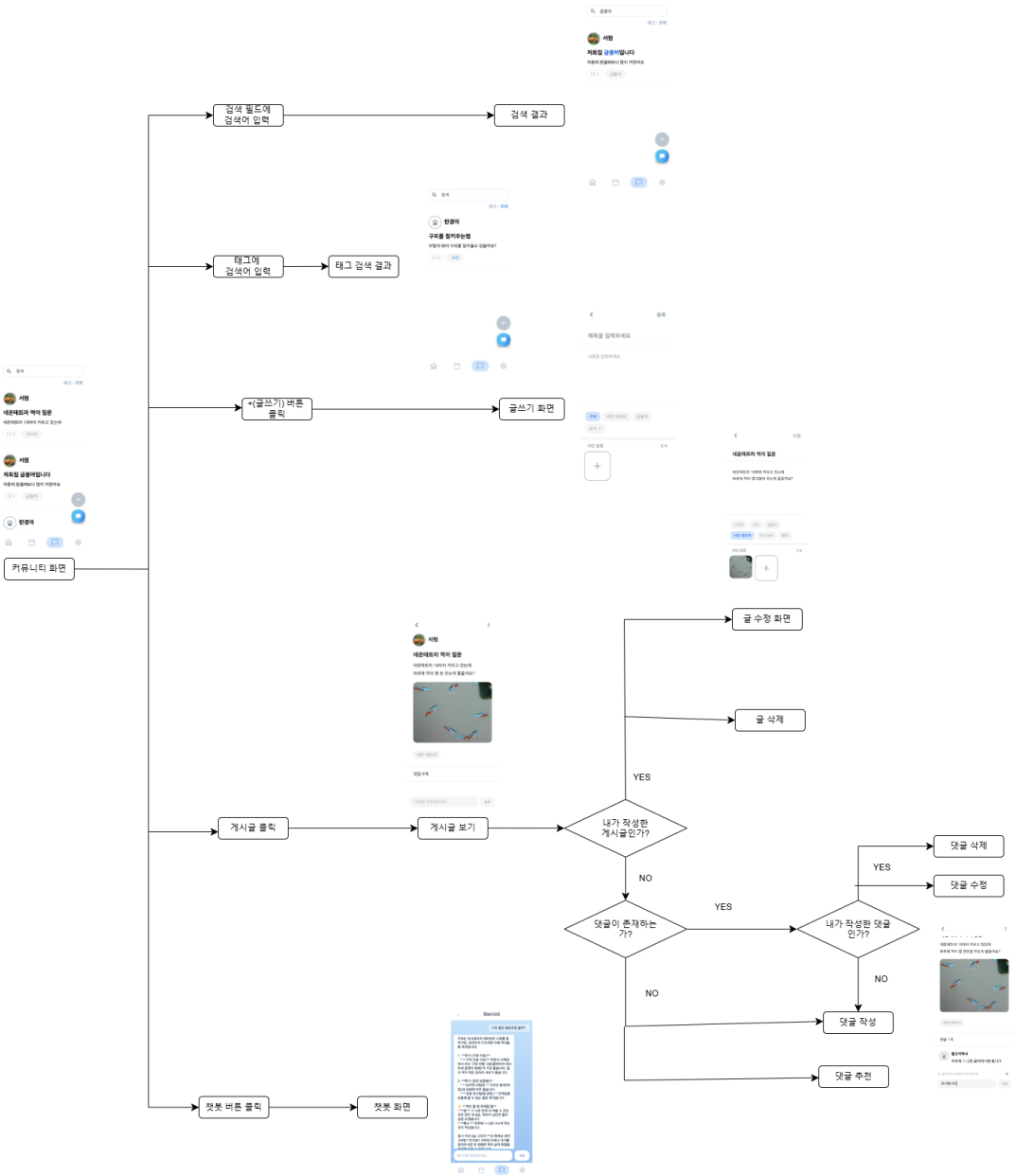
2.2. UI 설계


알람 버튼 클릭
알람 화면
카메라 버튼 클릭
카메라 화면
홈 화면
수온 버튼 클릭
수온 상세 화면
환수하기
버튼 클릭
환수하기 로딩화면
먹이주기
버튼 클릭
먹이주기 로딩화면


Login
시작 화면
카카오 로그인
구글 로그인
첫 로그인시
(회원가입시)
YES
NO
닉네임이
중복되는가?
YES
NO
닉네임 재입력
어항 등록하기
버튼 클릭
어항 시리얼 넘버,
최저 수온, 최고 수온,
최저 수질, 최고 수질
입력하여 어항 등록
홈 화면


설정 화면
프로필 변경
버튼 클릭
프로필 변경
화면
닉네임이
중복되는가?
NO
YES
저장 버튼 클릭
프로필 변경 완료
닉네임 재입력
나의 물고기
버튼 클릭
나의 물고기
화면
갱신하기
버튼 클릭
수정 버튼 클릭
나의 물고기
수정 화면
나의 물고기
종류 수정
나의 물고기
마릿수 수정
나의 물고기 삭제
어항 설정
버튼 클릭
어항 시리얼 넘버,
최저 수온, 최고 수온,
최저 수질, 최고 수질
입력하여 어항 설정 수정
로그아웃
버튼 클릭
시작 화면
Login


October
2026.10.27 기록
October
2026.10.17 기록
캘린더 화면
원하는 날짜
클릭
YES
과거 수조
관리 기록 확인
NO
평균 수온 클릭
캘린더 수온
기록 상세 화면


커뮤니티 화면
검색 필드에 검색어 입력
검색 결과
태그에 검색어 입력
태그 검색 결과
+(글쓰기) 버튼 클릭
글쓰기 화면
게시글 클릭
게시글 보기
내가 작성한 게시글인가?
YES
NO
글 수정 화면
글 삭제
댓글이 존재하는가?
YES
NO
내가 작성한 댓글인가?
YES
NO
댓글 삭제
댓글 수정
댓글 작성
댓글 추천
챗봇 버튼 클릭
챗봇 화면

2.3. 시스템 상세설계

2.4. 시스템 기능별 요약

기능	설명
로그인	해당 시스템을 이용하기 위해 사용자 인증을 실행한다.
사용자 정보 관리	닉네임, 프로필 사진 및 어항 설정 기능을 구현한다.
서버 및 데이터베이스	사용자 정보와 어항 데이터를 저장하고, 앱과 IoT 및 AI 서버 간의 통신을 처리한다.
커뮤니티	게시글과 댓글을 통해 사용자 간 정보를 공유할 수 있도록 한다.
IoT 센서 측정	센서를 통해 어항의 수온과 수질을 측정한다.
원격 먹이 급여 및 환수	먹이 급여와 환수를 원격으로 실행할 수 있도록 한다.
실시간 영상 스트리밍	사용자들끼리 관상어 정보를 공유하는 커뮤니티 환경을 조성한다.
AI 어종 인식	YOLO를 활용하여 관상어의 종류와 개체 수를 인식한다.
AI 활동성 이상 탐지	객체 추적 및 SOM을 활용하여 관상어의 활동성 이상 여부를 분석한다.
수조 상태 모니터링	수온·수질 조회, 기록 및 이상 알림 기능을 제공한다.
이상 알람	온도나 수질에 이상이 생기면 사용자에게 이상 알람을 제공한다.
AI 챗봇	LLM 기반 챗봇을 통해 관상어 관리 정보를 제공한다.
일일 기록 조회	선택한 날짜의 어항 상태와 관리 기록을 확인한다.

3. 결론 및 향후과제

3.1. 작품보완점 및 목표구현 정도

3.1.1 작품 보완점

-기기 번호를 직접 입력하는 등록 방식을 개선하여 보다 간편하게 IoT 기기를 등록할 수 있도록 한다.
-한 계정에서 여러 개의 어항을 등록하고 관리할 수 있도록 기능을 확장한다.
-AI 챗봇의 API 사용 비용을 줄여 경제적인 서비스 운영이 가능하도록 개선한다.
-관상어의 활동 데이터를 추가로 확보하여 활동성 이상 탐지의 정확도와 신뢰성을 개선한다.

3.1.2 목표구현

기능	설명	구현 여부 (O/X)
로그인 및 회원 관리	사용자 인증 및 회원가입 기능을 제공한다.	O
개인 정보 수정	사용자의 닉네임와 프로필 사진을 수정한다.	O
어항 정보 설정	어항 내 온도와 수질에 대한 기본값을 설정하거나 수정한다.	O
커뮤니티	사용자 간 관상어 정보를 공유할 수 있도록 한다.	O
IoT 센서 측정	센서를 통해 수온과 수질을 측정한다.	O
원격 먹이 급여 및 환수	먹이 급여와 환수를 원격으로 실행한다.	O
실시간 영상 스트리밍	카메라를 통해 어항 내부를 실시간으로 보여준다.	O
AI 어종 및 개체 수 인식	YOLO를 활용하여 관상어의 종류와 마리 수를 인식한다.	O
AI 활동성 이상 탐지	관상어의 움직임을 분석하여 활동성 이상 여부를 판단한다.	O
서버 및 데이터베이스	사용자 및 어항 데이터를 저장하고 시스템 간 통신을 처리한다.	O

이상 알림	수온 및 수질 이상 발생 시 알림을 제공한다.	O
AI 챗봇	관상어 관리에 필요한 정보를 제공한다.	O
일일 기록 조회	날짜별 어항 상태 및 관리 기록을 확인한다.	O

3.2. 개발 환경 및 도구

-Visual Studio Code(IDE)
-Flutter / Dart
-Python / FastAPI
-Node.js
-MySQL
-Raspberry Pi
-MediaMTX / WebRTC
-YOLO11 / OpenCV / SOM
-AWS(Amazon Web Service)
-Docker
-Git / GitHub

3.3. 향후 발전 방향

-커뮤니티에 친구 추가 및 개인 메시지 기능을 도입하여 사용자 간 소통과 정보 공유를 더욱 활성화한다.
-AI 분석 기능을 고도화하여 관상어의 건강 상태를 더욱 정확하게 파악할 수 있도록 한다.
-여러 개의 어항을 하나의 계정에서 통합 관리할 수 있도록 기능을 확장한다.

3.4. 문제점 및 개선 방안

-AI 챗봇의 API 사용량에 따라 비용이 발생하는 문제를 해결하기 위해 사용량을 최적화하고, 향후 일부 기능에 유료 서비스를 도입하는 방안을 고려한다.
-커뮤니티에서 부적절한 태그 사용을 방지하기 위해 미리 지정된 태그만 선택할 수 있도록 제한하였다. 향후 관리자 검토 및 승인 기능을 추가하여 사용자가 원하는 태그를 자유롭게 등록할 수 있도록 개선한다.

4. 실행 매뉴얼: 소스 코드 실행 환경 및 실행 방법

백엔드

프로젝트 구조

```
fishTankServer/
├── bin/
│   └── www                    # HTTP 서버 실행 및 WebSocket 연결
│
├── config/
│   └── config.js              # Sequelize 데이터베이스 연결 설정
│
├── models/                    # Sequelize 모델 및 테이블 관계 정의
│   ├── index.js               # 모델 초기화 및 연관 관계 설정
│   ├── user.js                # 사용자
│   ├── userimage.js           # 프로필 이미지
│   ├── tank.js                # 어항 및 관리 기준값
│   ├── sensor.js              # 수온 기록
│   ├── waterquality.js        # 수질 기록
│   ├── daily.js               # 일일 요약
│   ├── FeederLog.js           # 먹이 지급 기록
│   ├── waterchange.js         # 환수 기록
│   ├── alert.js               # 경고 알림
│   ├── Fishinfo.js            # 사육 중인 물고기 정보
│   ├── post.js                # 커뮤니티 게시글
│   ├── image.js               # 게시글 이미지
│   ├── comment.js             # 댓글 및 답글
│   ├── commentLike.js         # 댓글 좋아요
│   ├── chatbotRoom.js         # 챗봇 대화방
│   ├── chatbotmessage.js      # 챗봇 대화 기록
│   └── aiAnalyze.js           # AI 분석 모델
│
├── migrations/                # 데이터베이스 스키마 변경 이력
│
├── routes/                    # HTTP API 라우터
│   ├── index.js               # 기능별 라우터 연결
│   ├── auth/
│   │   ├── index.js           # 소셜 로그인, 토큰 재발급, 프로필 관리
│   │   └── authMiddleware.js  # JWT 액세스 토큰 검증
│   │   └── tank/
```

```
├── routes/                    # HTTP API 라우터
│   ├── index.js               # 기능별 라우터 연결
│   ├── auth/
│   │   ├── index.js           # 소셜 로그인, 토큰 재발급, 프로필 관리
│   │   └── authMiddleware.js  # JWT 액세스 토큰 검증
│   ├── tank/
│   │   ├── index.js           # 어항 설정, 센서 수신, 알림 조회
│   │   ├── tanksse.js         # 센서 캐시, 최고·최저 수온, SSE 및 경고 처리
│   │   └── schedual.js        # 센서·수질 주기 저장 및 일일 요약
│   ├── daily/
│   │   └── index.js           # 날짜별 일지 조회
│   ├── fishinfo/
│   │   └── index.js           # 물고기 정보 조회 및 저장
│   ├── communty/
│   │   └── index.js           # 게시글, 댓글, 좋아요 관리
│   ├── chatbot/
│   │   ├── index.js           # 대화방 및 메시지 API
│   │   └── chatbotSetting.js  # AI 답변 생성
│   └── uploaded/
│       ├── userupload.js      # 프로필 이미지 업로드 설정
│       └── profileupload.js   # 게시글 이미지 업로드 설정
│
├── socket/                    # WebSocket 통신 및 명령 처리
│   ├── index.js               # 사용자·IoT·AI 연결 등록 및 메시지 분기
│   ├── feed.js                # 먹이 지급 명령 및 결과 처리
│   ├── waterChange.js         # 환수 명령 및 결과 처리
│   └── Ai/
│       ├── userCommand.js     # AI 서버에 어종 분석 요청
│       └── AiResult.js        # 어종·활동 분석 결과를 사용자에게 전달
│
├──uploads/                    #업로드 이미지 저장소
│  ├──post                     #게시판에 올릴 사진 저장소
│  └──user                     #유저 프로필 올릴 사진 저장소
│
├── app.js                     # Express 앱, 미들웨어 및 라우터 설정
├── Dockerfile                 # 서버 컨테이너 이미지 빌드 설정
├── package.json               # 의존성 및 실행 스크립트
├── package-lock.json          # 의존성 버전 고정
└── README.md                  # 프로젝트 소개 및 실행 안내
```

실행 방법

1. 프로젝트 다운로드 하기

git clone https://github.com/ysy03/fishTankServer.git
cd fishTankServer

2. 환경변수 설정

.env파일 만든 다음 해당 내용 입력

```
DB_USERNAME = <DB_USERNAME>
DB_PASSWORD = <DB_PASSWORD>
DB_DATABSENAME = <DB_DATABASENAME>
DB_HOST = <DB_HOST>

chatbot_apiKey = <GEMINI_API_KEY>

JWT_SECRET = <JWT_SECRET>
JWT_REFRESH = <JWT_REFRESH>
```

3.실행 방법

서버는 크게 Node.js환경와 Docker환경으로 실행할 수 있습니다.

3-1.Node.js 환경

```
npm install
npm run start
```

3-2.Docker 환경

`docker build -t <Project-Name> .`
build:프로젝트가 돌아가는 환경을 이미지형태로 제작합니다.
-t:프로젝트 이름을 지정해줍니다.
'.':이미지에 들어갈 재료를 표시해줍니다.

`docker run -d <Project-Name> --env-file .env -p <로컬환경 포트번호>:<설정한 포트번호>`
run:제작한 이미지를 바탕으로 실행하는 환경인 컨테이너를 제작한 후 실행하게 합니다.
-p:포트번호를 지정해줍니다.
-d:백그라운드 환경에서 돌아가게 합니다.
--env-file:환경설정 파일을 가져와줍니다.

프론트엔드

프로젝트구조

```
lib/
├── api/                                    # 서버 API 통신 관련 코드
│   ├── api_client.dart                     # API 요청 및 응답 처리
│   ├── dairy_api.dart                      # 캘린더 API 통신
│   ├── motor_api.dart                      # 환수/급여 모터 API 통신
│   └── sensor_api.dart                     # 센서 데이터 API 통신
│
├── models/                                 # 데이터 모델 정의
│   ├── alert.dart                          # 알림 데이터 모델
│   ├── comment.dart                        # 커뮤니티 댓글 데이터 모델
│   ├── fishinfo.dart                       # 물고기 정보 데이터 모델
│   ├── post.dart                           # 커뮤니티 게시글 데이터 모델
│   ├── post_detail.dart                    # 게시글 상세 정보 데이터 모델
│   └── sensor.dart                         # 센서 데이터 모델
│
├── screens/                                # 앱 화면 및 페이지 UI
│   │
│   ├── community/                          # 커뮤니티 관련 화면
│   │   ├── community_detail_screen.dart    # 커뮤니티 게시글 상세 화면
│   │   ├── post_card.dart                  # 게시글 카드 UI 컴포넌트
│   │   └── post_writing_screen.dart        # 게시글 작성 화면
│   │
│   ├── iot/                                # IoT 및 어항 관리 관련 화면
│   │   ├── alert_screen.dart               # 알림 목록 화면
│   │   ├── daily_temp_detail_screen.dart   # 캘린더 수온 상세 정보 화면
│   │   ├── feeding_loading_screen.dart     # 먹이 급여 진행 화면
│   │   ├── fish_info_management_screen.dart # 나의 물고기 정보 관리 화면
│   │   ├── live_video_screen.dart          # 실시간 어항 영상 화면
│   │   ├── water_change_loading_screen.dart # 환수 진행 화면
│   │   └── water_temp_detail_screen.dart   # 수온 상세 정보 화면
│   │
│   ├── setting/                            # 설정 및 프로필 관련 화면
│   │   ├── myfish_screen.dart              # 나의 물고기 정보 관리 화면
│   │   └── profile_edit_screen.dart        # 프로필 수정 화면
│   │
│   ├── calendar_screen.dart                # 어항 관리 기록 캘린더 화면
│   ├── chatbot_screen.dart                 # AI 챗봇 화면
│   ├── community_list_screen.dart          # 커뮤니티 게시글 목록 화면
│   ├── home_screen.dart                    # 메인 홈 화면
│   ├── make_profile_screen.dart            # 프로필 생성 화면
│   ├── mypage_screen.dart                  # 마이페이지 화면
│   └── start_screen.dart                   # 앱 시작 화면
│
├── services/                               # 비즈니스 로직 및 외부 서비스 연동
│   ├── alert_service.dart                  # 알림 데이터 조회
│   ├── community_service.dart              # 커뮤니티 게시글 및 댓글 관련 기능
│   ├── daily_service.dart                  # 캘린더 어항 관리 데이터 조회
│   ├── fishinfo_service.dart               # 나의 물고기 정보 조회 및 관리
│   ├── socket_service.dart                 # 실시간 소켓 통신 관리
│   ├── tank_service.dart                   # 어항 정보 및 제어 관련 기능
│   └── token_storage.dart                  # 사용자 인증 토큰 저장 및 관리
│
├── widgets/                                # 재사용 가능한 UI 컴포넌트
│   ├── common_bottom_nav_bar.dart          # 공통 하단 네비게이션 바
│   ├── error_banner.dart                   # 오류 및 경고 메시지 표시 UI
│   ├── rotating_loading_icon.dart          # 회전형 로딩 아이콘
│   ├── tank_register.dart                  # 어항 등록 UI 컴포넌트
│   └── yes_or_no.dart                      # 예/아니요 선택 UI 컴포넌트
│
└── main.dart                               # 앱 실행 및 초기 설정
```

🚀 실행 방법

1️⃣ 프로젝트 클론

```
git clone https://github.com/ksw2003/ai_fishapp.git
cd ai_fishapp/
```

2️⃣ .env 파일 생성

프로젝트 루트 디렉토리에 .env 파일을 생성합니다.

```
cp env-template .env
```

3️⃣ .env 파일 생성

.env 파일에 다음 키 값들을 입력하세요:

```
KAKAO_NATIVE_APP_KEY= <KAKAO_NATIVE_APP_KEY>
GOOGLE_KEY= <GOOGLE_KEY>
API_BASE_URL= <BACKEND_SERVER_URL>
VIDEO_BASE_URL= <MEDIAMTX_SERVER_URL>
```

4️⃣ 앱 실행

환경 변수 설정이 완료되면 Flutter 앱을 실행할 수 있습니다.

```
flutter pub get
flutter run
```

AI백엔드

📁 프로젝트 구조

```
fish-ai-server/
├── core/
│   └── yolo_priority.py            # YOLO 분석 우선순위 제어
├── services/
│   ├── activity_result_service.py  # 활동성 분석 결과 처리
│   ├── activity_service.py         # SOM 기반 활동성 이상 탐지
│   └── species_service.py          # 어종 및 개체 수 분석
├── trackers/
│   └── custom_bytetrack.yaml       # ByteTrack 추적 설정
├── vision/
│   ├── activity_visualizer.py      # 활동성 추적 결과 시각화
│   ├── detector_csrt.py            # YOLO + CSRT 기반 물고기 추적
│   ├── detector.py                 # YOLO 기반 객체 탐지
│   └── rtsp_camera.py              # RTSP 실시간 영상 수신
├── websocket/
│   └── ws_client.py                # Node.js 서버 WebSocket 통신
├── weights/
│   ├── best.pt                     # 학습된 YOLO11n 모델
│   └── som_model.npz               # 학습된 SOM 모델
├── .env.example                    # 환경 변수 예시
├── .gitignore                      # Git 제외 설정
├── config.py                       # AI 서버 환경 설정
├── main.py                         # FastAPI 서버 실행 및 초기화
├── requirements.txt                # Python 패키지 목록
└── README.md                       # 프로젝트 설명 및 실행 방법
```

🚀 실행 방법

1️⃣ 프로젝트 클론

```
git clone https://github.com/dakyung427/fish-ai-server.git
cd fish-ai-server
```

2️⃣ 가상환경 생성 및 활성화

Python 3.11 및 Windows PowerShell 환경을 기준으로 합니다.

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

3️⃣ 패키지 설치

```
pip install -r requirements.txt
```

4️⃣ 환경 변수 설정

프로젝트 루트 디렉터리에 `.env` 파일을 생성합니다.

```
Copy-Item .env.example .env
```

`.env` 파일에 다음 항목을 설정합니다.

```
RTSP_URL=rtsp://<카메라서버주소>:8554/cam
NODE_WS_URL=ws://<백엔드서버주소>:3000
YOLO_MODEL_PATH=weights/best.pt
BYTETRACK_CONFIG=trackers/custom_bytetrack.yaml
```

- `RTSP_URL` : Raspberry Pi에서 송출하는 실시간 영상 주소
- `NODE_WS_URL` : Node.js 백엔드 WebSocket 서버 주소
- `YOLO_MODEL_PATH` : 학습된 YOLO 모델 경로
- `BYTETRACK_CONFIG` : ByteTrack 설정 파일 경로

5️⃣ AI 서버 실행

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

IOT

📁 프로젝트 구조

```
streaming/
├── mediamtx.yml        # MediaMTX 영상 송출 설정
├── mediamtx.service    # MediaMTX 자동 실행 서비스
├── aws-relay.service   # AWS 영상 중계 자동 실행 서비스
├── relay.sh            # FFmpeg 기반 RTSP 영상 중계
└── .env.example        # AWS 연결 환경 변수 예시

iot/
├── venv/                 # Python 가상환경 (Git 제외)
├── .env                  # 환경 변수 설정 파일 (Git 제외)
├── .env.example          # 환경 변수 설정 예시
├── requirements.txt      # Python 패키지 및 라이브러리 의존성 목록
├── config.py             # 공통 환경 변수 및 서버 설정 관리
├── feeder.py             # 먹이 급여 모터 제어
├── iot_ws.py             # 서버와 실시간 WebSocket 통신
├── main.py               # IoT 시스템 실행 및 전체 기능 관리
├── sensorAPI.py          # 센서 데이터 서버 전송
├── temperature.py        # 수온 센서 측정
├── waterPump.py          # 환수 펌프 제어
└── waterQuality.py       # 수질 데이터 측정 및 처리
```

⚙️ 실행 방법

🖥️ IoT

1 프로젝트 클론

```
git clone https://github.com/ksw2003/fishapp_iot.git
cd fishapp_iot
```

2 Python 가상환경 생성 및 활성화

```
python3 -m venv venv
source venv/bin/activate
```

3 필수 패키지 설치

```
pip install -r requirements.txt
```

4 환경 변수 설정

프로젝트 루트 디렉터리에 `.env` 파일을 생성하고 서버 연결 정보를 설정합니다.

```
SERVER_URL=<BACKEND_SERVER_URL>
SERVER_WS=<BACKEND_WS_SERVER_URL>
DEVICE_ID=SS501
```

5 IoT 프로그램 실행

가상환경이 활성화된 상태에서 메인 프로그램을 실행합니다.

```
python3 main.py
```

6 프로그램 종료

실행 중인 터미널에서 `Ctrl + C`를 입력하여 프로그램을 종료합니다.

🎥 실시간 영상 송출

1️⃣ MediaMTX 및 FFmpeg 설치

Raspberry Pi 환경에 MediaMTX와 FFmpeg를 설치합니다.

MediaMTX는 공식 GitHub 저장소에서 다운로드할 수 있습니다.

2️⃣ 설정 파일 배치

파일	Raspberry Pi 경로
`mediamtx.yml`	`/home/fish/mediamtx.yml`
`mediamtx.service`	`/etc/systemd/system/mediamtx.service`
`aws-relay.service`	`/etc/systemd/system/aws-relay.service`
`relay.sh`	`/home/fish/aws-relay/relay.sh`
`.env`	`/home/fish/aws-relay/.env`

서비스 파일에 지정된 사용자 이름과 실행 경로는 Raspberry Pi 환경에 맞게 수정해야 합니다.

3️⃣ 환경 변수 설정

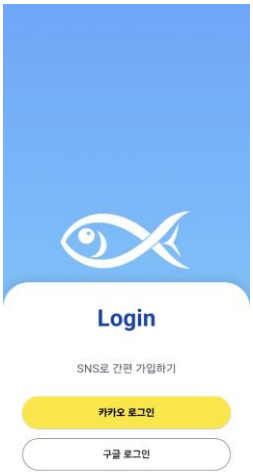
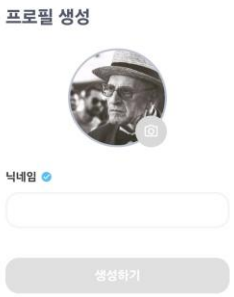
`streaming/.env.example` 을 참고하여 `/home/fish/aws-relay/.env` 파일을 생성합니다.

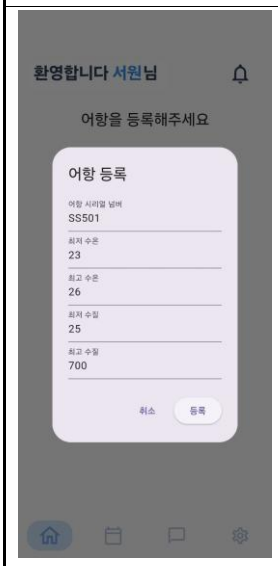
```
AWS_PUBLIC_IP= <YOUR_AWS_PUBLIC_IP>
RTSP_USER= <FISHPUBLISHER>
RTSP_PASSWORD= <YOUR_RTSP_PASSWORD>
```

AWS EC2의 MediaMTX 서버에 설정된 RTSP 인증 정보를 입력합니다.

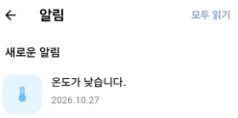
5. 사용자 매뉴얼: 작품의 사용 방법

1. 회원가입 후 홈화면 이동

1. 시작 화면	2. 로그인 화면	3. 로그인 후 프로필 생성	4. 어항 등록 버튼 클릭
			

5. 어항 정보 등록	6. 정보등록 후 홈화면 이동
	

2. 어항 관리 서비스 이용

1. 홈 화면 설명	2. 카메라 버튼을 클릭하여 실시간 어항 영상 및 관상어 건강/마릿수/종류 AI분석 결과 확인	3. 알림 버튼을 클릭하여 수온/수질 이상 확인	3. 수온 버튼을 클릭하여 현재 수온과 오늘의 최고·최저 수온 정보를 확인
			

5. 환수하기 버튼을 클릭하여 원격 환수 실행	6. 먹이주기 버튼을 클릭하여 원격 급여 장치 작동
	

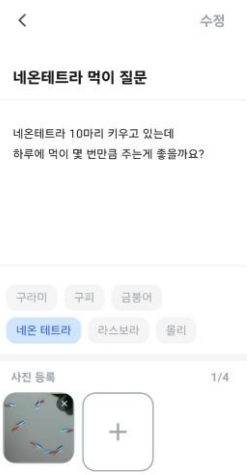
3. 캘린더 어플 관리 일지 확인

1. 캘린더 화면에서 원하는 날짜를 선택하여 그날의 수온·수질 정보와 급여 및 환수 기록을 확인	2. 평균 수온 버튼을 클릭해 해당 날짜의 평균 수온과 최고·최저 수온을 확인
	

4. 커뮤니티 글쓰기

1. 커뮤니티 화면에서 우측 하단의 “+” 버튼을 클릭	2. 게시글 작성 화면에서 제목과 내용, 태그와 사진 등록	3. 상단 등록 버튼 클릭 후 게시글 등록 완료
		

5. 내가 쓴 글 수정 및 삭제

1. 자신이 작성한 게시글의 우측 상단 더보기 버튼을 클릭해 수정/삭제 버튼을 선택	2-1. 내용/태그 혹은 사진 수정 후 수정 버튼 클릭	3. 저장이 완료되면 이전 화면으로 이동	2-2. 삭제가 완료되면 커뮤니티 게시판 화면으로 이동
			

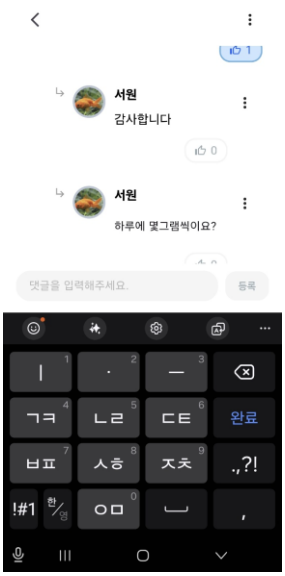
6. 댓글 작성

1. 글 아래의 '댓글을 입력해주세요' 필드 선택	2. 댓글 입력창에 내용을 작성 후 등록 버튼 클릭시 댓글이 게시글에 추가
	

7. 답글 작성 및 댓글 추천

1. 게시글의 댓글을 선택 후 내용 작성	2. 등록 버튼을 누르면 해당 댓글 아래에 답글이 추가됨	3. 댓글 추천 버튼 클릭으로 추천 가능
		

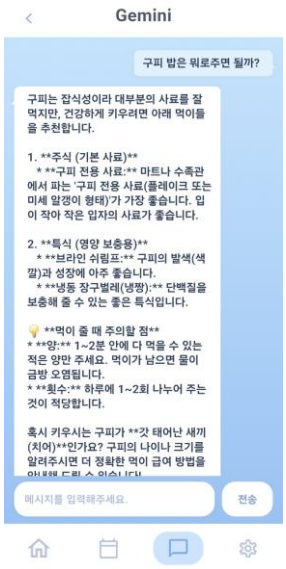
8. 댓글/답글 수정 및 삭제

1. 자신이 작성한 댓글 또는 답글 상단의 더보기 버튼 클릭	2. 수정 버튼을 선택해 댓글 수정 후 등록 버튼 클릭	3. 삭제버튼 클릭시 댓글 삭제
		

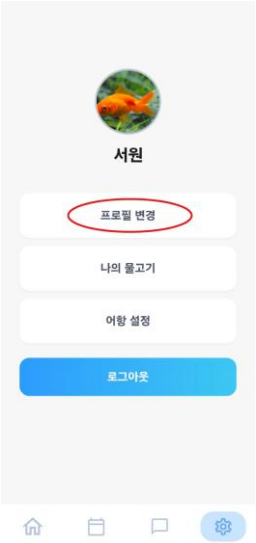
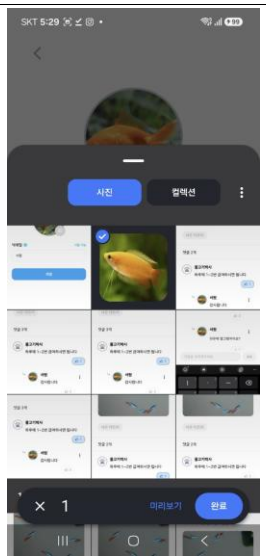
9. 게시글/태그 검색

1. 상단 검색 필드, 검색 필드 아래의 태그 필드를 이용해 검색	2-1. 검색어 입력 후 검색 결과	2-2. 태그 입력 후 검색 결과
		

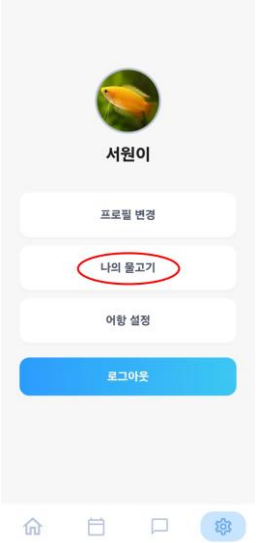
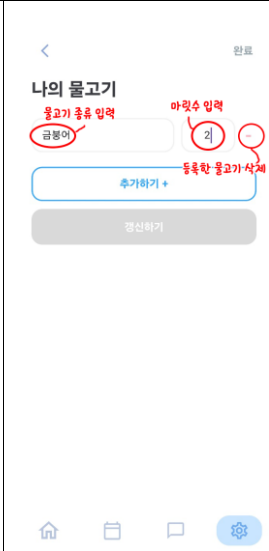
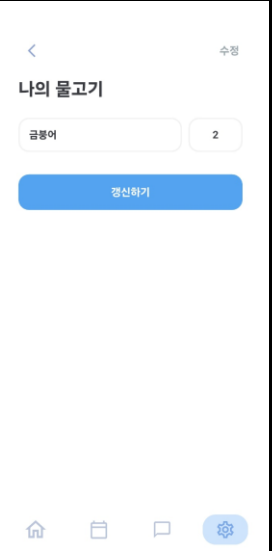
10. 챗봇 사용

1. 커뮤니티 화면에서 우측 하단의 파란색 말풍선 버튼을 클릭	2. 챗봇 상담 화면으로 이동
	

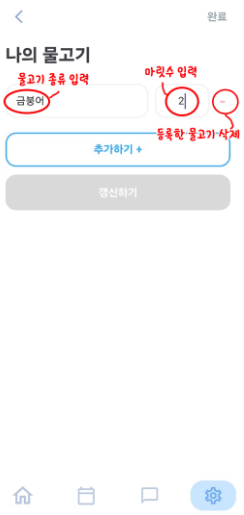
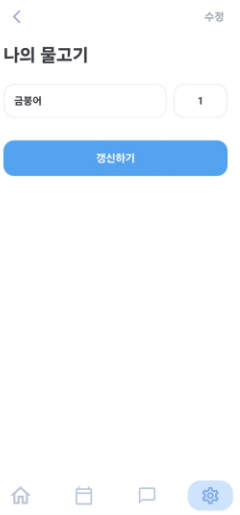
11. 프로필 변경

1. 마이페이지 화면에서 프로필 변경 버튼을 클릭	2. 변경할 닉네임 입력	3. 프로필 사진 바꾸기	4. 저장버튼 클릭 후 프로필 변경 완료
			

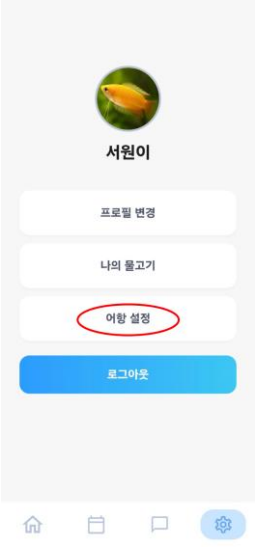
12. 나의 물고기 추가

1. 마이페이지 화면에서 “나의 물고기” 버튼을 클릭	2. 추가하기 버튼 클릭 후 물고기 종류와 마릿수 작성	3. 완료버튼 클릭시 작성된 나의 물고기가 저장됨
		

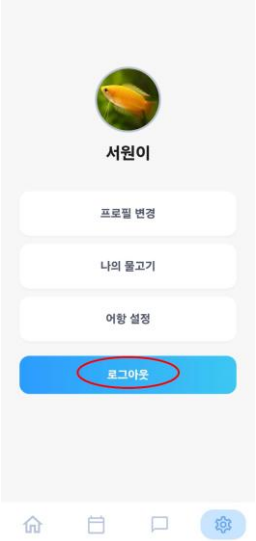
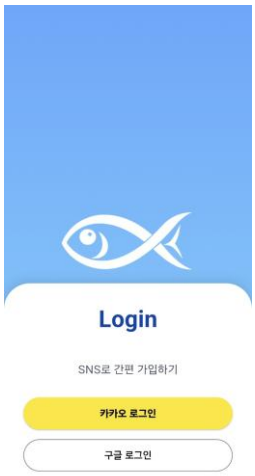
13. 나의 물고기 수정 및 삭제

1. 우측 상단의 수정 버튼 클릭	2. 물고기 종류 혹은 마릿수 변경 및 삭제 가능	3. 완료버튼 클릭시 수정 완료	4. 삭제 버튼 클릭시 등록된 물고기 삭제
			

14. 어항 설정 변경

1. 사용자가 마이페이지 화면에서 “어항 설정” 버튼을 클릭	2. 현재 설정된 어항 정보 확인	3. 어항 설정 변경 후 변경 버튼 클릭
		

15. 로그아웃

1. 마이페이지 화면에서 “로그아웃” 버튼을 클릭	2. 로그아웃이 진행되며 사용자 세션 정보가 초기화됨	3. 로그아웃 완료 후 로그인 화면으로 이동
		

부 록

R-1. 참고문헌 및 참고사이트

YOLO 학습 데이터셋

[1]https://universe.roboflow.com/zehra-acer/fish-detection-fztlb

R-2. 프로그램 소스(소프트웨어 개발 시)

GitHub

프로젝트 대표저장소

[1]https://github.com/ksw2003/fishapp_main

백엔드

[2]https://github.com/ysy03/fishTankServer

프론트엔드

[3]https://github.com/ksw2003/ai_fishapp

AI백엔드

[4]https://github.com/dakyung427/fish-ai-server

IOT

[5]https://github.com/ksw2003/fishapp_iot

소 감

김서원

이번 프로젝트는 다양한 기술을 직접 다루고 통합하여 실제 서비스로 구현해 볼 수 있었던 의미 있는 경험이었다.
라즈베리파이를 처음 다뤄보았기 때문에 겪는 어려움도 있었지만, 새로운 기술을 하나씩 익혀가며 문제를 해결하는 과정에서 성취감을 느낄 수 있었다. 또한 UI를 구성하고 각 기능이 실제 데이터와 연동되도록 구현하는 과정에서 발생한 문제들을 조원들과 함께 해결해 나가며 협력의 가치를 배울 수 있었다.

유승엽

이번 관상어용 스마트 어항 프로젝트를 진행하며 팀원들과 소통하고 협력하는 과정의 중요성을 배울 수 있었다. 회의에서 각자 발견한 문제와 의견을 공유하면서 혼자서는 생각하지 못했던 문제점을 파악하고, 함께 해결 방안을 찾아갈 수 있었다. 이를 통해 하나의 문제를 다양한 관점에서 바라보고 해결하는 시야를 넓힐 수 있었다. 또한 Docker를 활용해 프로젝트를 관리하고 배포하는 방법을 직접 익히면서, 개발한 시스템이 실제 환경에서 동작하는 과정에 재미와 성취감을 느꼈다.

이다경

이번 프로젝트를 통해 AI, 프론트엔드, 백엔드, IoT 등 서로 다른 기술을 하나의 시스템으로 통합하는 과정을9 경험할 수 있었다. 각 파트가 유기적으로 연동되도록 구현하면서 개별 기능뿐만 아니라 전체적인 시스템 구조와 흐름을 고려하는 것이 중요하다고 느꼈다. 특히 여러 기능을 하나로 통합하는 과정에서 팀원들과의 원활한 소통과 협력이 얼마나 중요한지도 깨달을 수 있었던 뜻깊은 경험이었다.